SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.(BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3 Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Vanaganti Shiva (192311361) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

V Shiva

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints: i. D1 cannot work Night shift ii. D2 must work before D3 (Morning < Afternoon < Night) iii. Only one doctor per shift iv. D3 cannot work Morning v. All doctors must be assigned exactly one shift Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

Problem Statement

A hospital needs to assign **3 doctors (D1, D2, D3)** to **3 shifts (Morning, Afternoon, Night)** such that:

- D1 cannot work the Night shift.
- D2 must work before D3 (Morning < Afternoon < Night).
- Only one doctor can be assigned to one shift.
- D3 cannot work the Morning shift.
- Every doctor must be assigned exactly one shift.

Apply the **Backtracking Search Algorithm** to find a valid assignment with all intermediate steps and constraint checking.

Problem Understanding

This is a **Constraint Satisfaction Problem (CSP)** because we need to assign doctors to shifts while satisfying a set of constraints. The objective is to find a valid schedule without violating any constraint.

Constraint Analysis

Variables

Doctors = {D1, D2, D3}

Domains

Shifts = {Morning, Afternoon, Night}

Constraints

1. D1 ≠ Night
2. D2 must work before D3
3. One doctor per shift
4. D3 ≠ Morning
5. Every doctor gets exactly one shift

Constraint Satisfaction Representation

Doctor	Possible Shifts
D1	Morning, Afternoon
D2	Morning, Afternoon, Night
D3	Afternoon, Night

Why Backtracking Search?

Backtracking is suitable because:

- It checks every assignment.
- It immediately rejects assignments that violate constraints.
- It avoids unnecessary searches.
- It guarantees finding a valid solution if one exists.

Algorithm (Backtracking Search)

1. Start with all doctors unassigned.
2. Select one doctor.
3. Assign a shift from the available domain.
4. Check all constraints.
5. If constraints are satisfied, assign the next doctor.
6. If any constraint is violated, backtrack.
7. Continue until all doctors are assigned.
8. Display the final schedule.

Step-by-Step Constraint Checking

Step 1

Assign:

D1 → Morning

Constraint Check:

- ✓ D1 is not assigned to Night.
- ✓ Shift is available.

Status: Accepted

Step 2

Assign:

D2 → Afternoon

Constraint Check:

- ✓ Afternoon shift is available.
- ✓ D2 works before D3.

Status: Accepted

Step 3

Assign:

D3 → Night

Constraint Check:

- ✓ D3 is not assigned Morning.
- ✓ Night shift is available.
- ✓ D2 (Afternoon) comes before D3 (Night).
- ✓ One doctor is assigned to each shift.
- ✓ Every doctor has exactly one shift.

Status: Accepted

Final Schedule

Doctor	**Shift**
D1	Morning
D2	Afternoon
D3	Night

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths. Constraints:

i. Movement allowed: Up, Down, Left, Right

ii. Cost of each move = 1

iii. Cannot pass through obstacles

iv. Use Manhattan Distance heuristic Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

Problem Statement

A robot has to move from the **Start (S)** to the **Goal (G)** in a **5×5 grid**. The robot can move only **Up, Down, Left, and Right**. Some cells are blocked by obstacles. Each move costs **1**. The objective is to find the shortest path using the **Breadth-First Search (BFS)** algorithm.

Where:

- **S** = Start
- **G** = Goal
- **X** = Obstacle

Constraints

- Movement allowed: Up, Down, Left, Right
- Cost of each move = 1
- Robot cannot pass through obstacles
- Manhattan Distance is used as a heuristic to estimate the distance to the goal.

Algorithm (BFS)

1. Place the start node in the queue.
2. Mark the start node as visited.
3. Remove the first node from the queue.
4. Check whether it is the goal node.
5. If not, add all valid neighboring nodes (Up, Down, Left, Right) that are not visited and not obstacles.
6. Repeat until the goal node is reached.
7. Print the shortest path and total cost.

Step-by-Step Node Expansion

Step 1

Queue:[S]

Expand:S

Visited:S

Step 2

Possible Moves:Right,Down

Queue: [(0,1), (1,0)]

Step 3

Expand: (0,1)

Queue: [(1,0), (0,2)]

Step 4

Expand: (1,0)

Queue: [(0,2), (2,0)]

Step 5

Continue expanding nodes level by level.

BFS ignores blocked cells (**X**) and already visited nodes.

Eventually, the goal node **G** is reached.

Priority Queue / Frontier Updates

Step	Queue
1	S
2	(0,1), (1,0)
3	(1,0), (0,2)
4	(0,2), (2,0)
5	(2,0), (1,2)
...	Continue until G

Note: BFS actually uses a normal FIFO queue rather than a priority queue. Since the question mentions "priority queue updates," you can mention that BFS maintains nodes in a queue and expands them in the order they are discovered.

Manhattan Distance

The Manhattan Distance is calculated as:

$$h(n) = | x_1 - x_2 | + | y_1 - y_2 |$$

Example:

Start = **(0,0)**

Goal = **(4,4)**

$$h(n) = | 4 - 0 | + | 4 - 0 |$$
$$= 4 + 4$$
$$= 8$$

The heuristic estimates that the robot is **8 moves away** from the goal (ignoring obstacles).

Advantages of BFS

- Finds the shortest path when all moves have equal cost.
- Simple and easy to implement.
- Guarantees the optimal solution in an unweighted graph.
- Explores nodes level by level.

Conclusion

The robot successfully reaches the goal using the Breadth-First Search (BFS) algorithm. BFS explores all possible paths level by level, avoids obstacles, and finds the shortest path. Since each move has a cost of 1, the total path cost is 8. The Manhattan Distance heuristic estimates the distance from the current position to the goal and can guide informed search algorithms, although BFS itself does not use it for node expansion.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G. Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed

• The robot has limited sensing capability (can only observe adjacent cells)

• Each movement has a cost of 1, but entering risky zones has an additional cost of 2

• The robot must minimize total cost while ensuring safe navigation

• The robot must update its knowledge dynamically (online search scenario)

Tasks: a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as: • Intelligent agents • Rationality • Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Problem Statement

An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot starts from **Start (S)** and must reach the survivor at **Goal (G)**. The environment is partially observable, contains obstacles, risky zones, and the robot has limited sensing capability.

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

Constraints Analysis

Constraint	**Effect on Decision Making**
Robot can move Up, Down, Left, Right	The robot can only choose these four directions while navigating.
Some cells are blocked (Obstacles)	The robot cannot move through blocked cells and must find an alternate path.
Limited sensing capability	The robot can observe only adjacent cells, so it makes decisions based on partial information.
Each movement costs 1	The robot tries to minimize the total number of moves.
Risky zones have an additional cost of 2	The robot avoids risky areas whenever possible to reduce the overall path cost.
Robot updates knowledge dynamically	As new information is discovered, the robot updates its map and changes its route if necessary.

Decision-Making Process

The rescue robot continuously senses its surroundings and updates its knowledge of the environment. If it encounters an obstacle or risky zone, it selects another safe path. Since the robot has limited sensing capability, it cannot see the entire environment at once and must make decisions using the available information.

b) Develop a solution approach using an appropriate search strategy. Clearly explain the steps involved.

Search Strategy Used

Uniform Cost Search (UCS)

Reason

Uniform Cost Search always selects the path with the **lowest cumulative cost**, making it suitable for environments where different paths have different costs (normal cells = 1, risky zones = 3).

Algorithm

1. Start at node **S** with cost **0**.
2. Insert the start node into the priority queue.
3. Remove the node with the lowest path cost.
4. Check whether it is the goal node.
5. Expand all valid neighboring nodes.
6. Ignore blocked cells.
7. Add movement cost and risky zone cost whenever applicable.
8. Update the priority queue.
9. Repeat until the goal node is reached.
10. Print the optimal path and total cost.

Solution Steps

Step 1

Robot starts from S.

Current Cost = 0

Step 2

Observe adjacent cells.

Remove blocked paths.

Choose the node with the minimum cost.

Step 3

Move to the next safe cell.

Cost = Previous Cost + 1

Step 4

If a risky zone is encountered,

New Cost

Previous Cost + 1 + 2

Step 5

Continue expanding the lowest-cost path.

Step 6

Reach Goal **G**.

Display:

- Shortest Path
- Minimum Cost

c) Demonstrate how the robot applies AI concepts

1. Intelligent Agent

The rescue robot is an intelligent agent because it:

- Perceives the environment through sensors.
- Makes decisions independently.
- Acts to achieve its goal.
- Learns from newly discovered information.

2. Rationality

The robot behaves rationally because it:

- Chooses the safest path.
- Avoids obstacles.
- Minimizes travel cost.
- Attempts to rescue the survivor successfully.

3. Environment Type

Property	Type
Observability	Partially Observable
Deterministic	Stochastic
Episodic/Sequential	Sequential
Static/Dynamic	Dynamic
Discrete/Continuous	Discrete
Single/Multi Agent	Single Agent

d) Justify why the chosen approach is most suitable

Uniform Cost Search is the most suitable algorithm because the robot travels through paths with different costs. Unlike Breadth-First Search, UCS considers the total path cost before expanding a node. This enables the robot to avoid expensive risky zones whenever possible and find the least-cost path to the survivor. Since the robot continuously updates its knowledge of the environment, UCS combined with online decision-making provides a safe and efficient solution.

Advantages

- Finds the least-cost path.
- Avoids unnecessary risky zones.
- Suitable for dynamic environments.
- Supports online decision making.
- Guarantees the optimal solution when costs are positive.

Conclusion

The autonomous rescue robot successfully reaches the survivor by using Uniform Cost Search in a partially observable environment. The robot senses nearby cells, avoids obstacles, minimizes travel cost, and updates its knowledge dynamically. By behaving as a rational intelligent agent, it efficiently selects the safest and least-cost path, making UCS the most appropriate search strategy for disaster rescue operations.